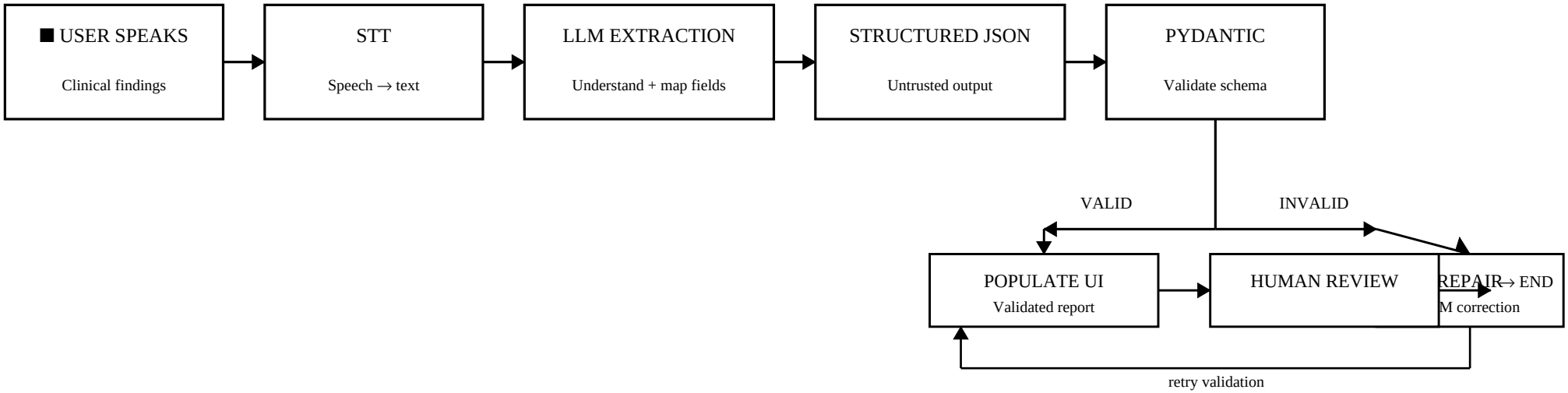
Pathology Voice Reporting — MVP Product Architecture

A focused clinical documentation prototype that converts a spoken pathology description into validated structured report fields, with a conditional LangGraph validation/repair loop before the report reaches the UI.



What you are building

The first product stage is a pathology voice-reporting assistant. A pathologist speaks naturally about a case. Speech-to-text converts the audio into a transcript. LangGraph then orchestrates an LLM extraction step that maps the transcript into a predefined pathology-report schema. The JSON is treated as untrusted until Pydantic validates it. If validation fails, a conditional edge sends the state to a repair node and then back through validation. Only validated data is sent to the UI, where the report fields are automatically populated for human review.

MVP stack

Layer	Technology	Role
API / Backend	FastAPI + Python	Receives audio, exposes workflow/report endpoints, connects UI to the LangGraph workflow.

Workflow	LangGraph	Stateful orchestration, nodes, conditional edges, validation/repair loop.
Schema	Pydantic	Defines the pathology report contract and validates LLM output.
LLM	Structured-output capable LLM	Extracts clinical information from the transcript into the report schema.
Speech-to-text	STT provider	Converts clinician speech into a text transcript.
Database	MySQL	Stores users/cases/reports and later workflow history.
UI	Simple web UI	Displays the structured report and allows human review/editing.

Initial LangGraph state

audio_path → source audio | **transcript** → STT result | **structured_report** → raw LLM JSON | **validated_report** → Pydantic-approved report | **error** → validation/processing error used by the conditional router.

Product boundary for this stage: the system is an assistive documentation prototype. It extracts and structures what the clinician says; it should not autonomously diagnose a patient. The clinician reviews and confirms the generated fields before finalizing the report.